All-Pairs Shortest Paths

1

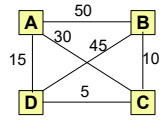
All-Pairs Shortest Paths

Problem

Given a ***weighted directed*** or ***undirected graph***, the problem is to find the shortest distances between ***all*** pairs of vertices in the graph.

- A simple approach to solving this problem involves computations of ***all possible distances*** through different paths, and then select the smallest computed distances.

- Consider the ***weighted undirected*** graph, with vertices *A,B,C,D*, shown in the figure.



Weighted undirected graph

- The possible paths between the vertices *A* and *B* , the associated distances for each the paths are as follows:

A->B = 50 (direct path)
A->D->B = 15 + 45 = 60 (one intermediate vertex)
A->C->B = 30 +10 =40 (one intermediate vertex)
A->D->C->B = 15 + 5 + 10 = 30 (two intermediate vertices)
A->C->D->B=30 + 5 + 45 = 80 (two intermediate vertices)

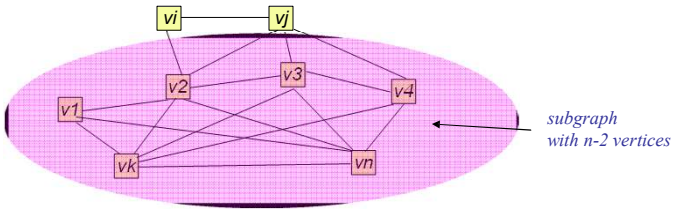
- It follows that the path ***A->D->C>-B*** is the ***shortest path*** between the vertices *A,B* with distance of *30*

2

All-Pairs Shortest Paths

Brute Force Algorithm

Let us consider an undirected graph with *n* vertices,*v1,v2,v3,.....vn*. The shortest distance between vertices ***vi*** and ***vj*** can be along the ***direct path*** or ***via intermediate paths*** connecting other ***n-2*** vertices.



Weighted graph with n vertices

▪Since *n-2* vertices can be permuted in *(n-2)!* ways, in order to determine the smallest distance via the intermediate paths, a total of *(n-2)!* distances would need to be computed. For large *n*, this would involve huge amount of computation. For example, if *n=30*, the number of computations would be of the order of $28! \approx 2.9x10^{29}$.

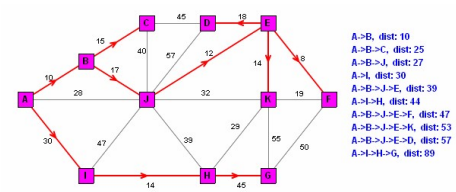
▪ Thus, brute force algorithm is ***highly inefficient***. Being of the order $\theta(n!)$, it is and even ***worse than exponential algorithm,*** and is not practical for the solution of large problems.

3

All-Pairs Shortest Paths

Other Algorithms

▪The ***Dijkstra's algorithm*** provides shortest distances from a ***given vertex*** to all other vertices.



Shortest distances between vertex A and other vertices , using Dijkstra's algorithm

▪In principle, ***Dijkstra's algorithm*** can be applied ***n times*** to compute the shortest distances between all pair of vertices in a graph. This approach is, however, inconvenient and not desirable.

▪An alternative more efficient algorithm is provided by ***Floyd*** and ***Warshall***, which systematically computes the distances between ***all pairs of vertices***. It is based on the dynamic programming approach. The shortest distances are stored in a matrix. Also, the the algorithm can be used to ***list the shortest paths***.

4

Floyd-Warshall Algorithm

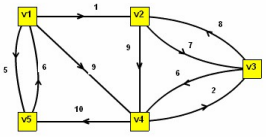
Weight Matrix

▪ The Floyd-Warshall algorithm uses a ***weight matrix*** to compute the intermediate shortest path in a directed or undirected graph.

▪ Let $G=(V,E)$ a weighted graph with weight $w(i,j)$ associated with the edge $e(v_i,v_j)$ between the vertices v_i, v_j. The ***weight matrix,*** w, with elements w_{ij} is defined as follows

$$w_{ij} = \begin{cases} 0 & \text{if } i = j \\ \infty & \text{if } e(v_i,v_j) \notin E \\ w(i,j) & \text{if } e(v_i,v_j) \in E \end{cases}$$

▪ The $w_{ij}=0$ implies that there is ***no loop*** at vertex vi, i.e, paths of zero length are not considered $w_{ij}=\infty$ implies that there is ***no direct path*** (edge) between the vertices v_i,v_j. The infinity symbol represent a very large value. Figure shows a directed weighted graph and associated weight matrix.



(a) Weighted graph

0	1	∞	9	5
∞	0	7	9	∞
∞	8	0	6	∞
∞	∞	2	0	10
6	∞	∞	∞	0

(b) Weight matrix

5

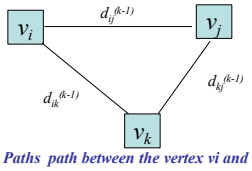
Floyd-Warshall Algorithm

Dynamic Programming Formulation

It follows from the preceding discussion that the matrix $D^{(k)}$ stores shortest distances which are computed by taking into the consideration the intermediate paths passing through the vertices $v1,v2,v3,....vk$.

▪ Let $d_{ij}^{(k)}$ be an element *of* $D^{(k)}$. In particular, $d_{ij}^{(0)}$ is equal to w_{ij} of ***weight matrix***

▪ The element $d_{ij}^{(k)}$ is computed by using the entries in the matrix $D^{(k-1)}$. If the shortest distance is along the direct path, then $\boldsymbol{d_{ij}^{(k)}=d_{ij}^{(k-1)}}$. However, if the shortest path is through an intermediate vertex, then $\boldsymbol{d_{ij}^{(k)}=d_{ik}^{(k-1)}+d_{kj}^{(k-1)}}$, as illustrated in the figure.



Paths path between the vertex vi and vj

▪ It therefore follows that the $d_{ij}^{(k)}$ satisfies the following recurrence:

$$d_{ij}^{(0)} = w_{ij}$$

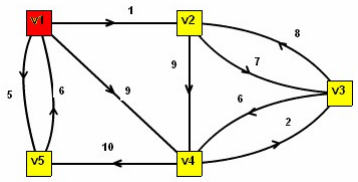
$$\underbrace{d_{ij}^{(k)}}_{\text{Shortest distance}} = \text{minimum}\left(\underbrace{d_{ij}^{(k-1)}}_{\text{Direct distance}}, \underbrace{d_{ik}^{(k-1)} + d_{kj}^{(k-1)}}_{\text{distance via vertices v1,v2,…vk}} \right)$$

6

Floyd Warshall Algorithm

Example-1

First, the shortest distances passing through vertex ***v1*** are computed, using the ***weight matrix***. The ***shortest distances*** are saved in the ***matrix $D^{(1)}$***. The steps are shown in the figures.



(a) Weighted graph

	v1	v2	v3	v4	v5
v1	0	1	∞	9	5
v2	∞	0	7	9	∞
v3	∞	8	0	6	∞
v4	∞	∞	2	0	10
v5	6	∞	∞	∞	0

(b) Weigh matrix

	v1	v2	v3	v4	v5
v1	0	1	∞	9	5
v2	∞	0	7	9	∞
v3	∞	8	0	6	∞
v4	∞	∞	2	0	10
v5	6	7	∞	15	0

(c) $D^{(1)}$ tabulates shortest distances through v1

Shortest paths through: v1

Direct distance, v5->v2 = ∞
Distance, via v5->v1->v2 = 7

Direct distance, v5->v4 = ∞
Distance, via v5->v1->v4 = 15

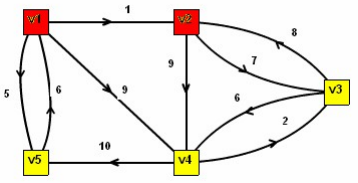
(d) Paths with shorter distances compared to direct paths

7

Floyd Warshall Algorithm

Example-2

The shortest distances passing through vertices ***v1, v2*** are computed, using the ***matrix $D^{(1)}$***
The ***shortest distances*** are saved in the ***matrix $D^{(2)}$***.



(a) Weighted graph

	v1	v2	v3	v4	v5
v1	0	1	∞	9	5
v2	∞	0	7	9	∞
v3	∞	8	0	6	∞
v4	∞	∞	2	0	10
v5	6	7	∞	15	0

(b) Matrix $D^{(1)}$ stores shortest distances through v1

	v1	v2	v3	v4	v5
v1	0	1	8	9	5
v2	∞	0	7	9	∞
v3	∞	8	0	6	∞
v4	∞	∞	2	0	10
v5	6	7	14	15	0

(c) $D^{(2)}$ tabulates shortest distances through v1,v2

Shortest paths through: v1 v2

Direct distance, v1->v3 = ∞
Distance, via v1->v2->v3 = 8

Direct distance, v5->v3 = ∞
Distance, via v5->v2->v3 = 14

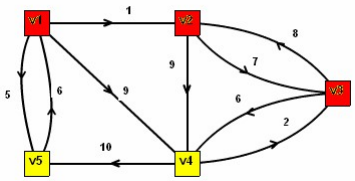
(d) Paths with shorter distances compared to direct distances

8

Floyd Warshall Algorithm

Example-3

The shortest distances passing through vertices ***v1,v2,v3*** are computed, using the ***matrix D(2)***
The ***shortest distances*** are saved in the ***matrix D(3).***



(a) Weighted graph

	v1	v2	v3	v4	v5
v1	0	1	8	9	5
v2	∞	0	7	9	∞
v3	∞	8	0	6	∞
v4	∞	∞	2	0	10
v5	6	7	14	15	0

(b) Matrix D(2) stores shortest distances through v1,v2

	v1	v2	v3	v4	v5
v1	0	1	8	9	5
v2	∞	0	7	9	∞
v3	∞	8	0	6	∞
v4	∞	10	2	0	10
v5	6	7	14	15	0

(c) D(3) tabulates shortest distances through v1,v2,v3

Shortest paths throuh: v1 v2 v3

Direct distance, v4->v2 = ∞
Distance, via v4->v3->v2 = 10

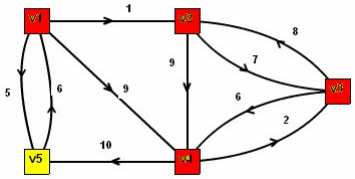
(d) Paths with shorter distances compared to direct distances

9

Floyd Warshall Algorithm

Example-4

The shortest distances passing through vertices ***v1,v2,v3,v4*** are computed, using the ***matrix D(3)***
The ***shortest distances*** are saved in the ***matrix D(4).***



(a) Weighted graph

	v1	v2	v3	v4	v5
v1	0	1	8	9	5
v2	∞	0	7	9	∞
v3	∞	8	0	6	∞
v4	∞	10	2	0	10
v5	6	7	14	15	0

(b) Matrix D(3) stores shortest distances through v1,v2,v3

	v1	v2	v3	v4	v5
v1	0	1	8	9	5
v2	∞	0	7	9	19
v3	∞	8	0	6	16
v4	∞	10	2	0	10
v5	6	7	14	15	0

(c) D(4) tabulates shortest distances through v1,v2,v3,v4

Shortest paths throuh: v1 v2 v3 v4

Direct distance, v2->v5 = ∞
Distance, via v2->v4->v5 = 19

Direct distance, v3->v5 = ∞
Distance, via v3->v4->v5 = 16

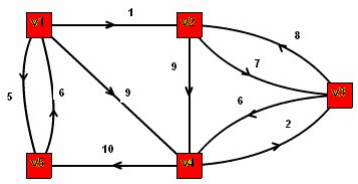
(d) Paths with shorter distances compared to direct distances

10

Floyd Warshall Algorithm

Example-5

The shortest distances passing through vertices ***v1,v2,v3,v4,v5*** are computed, using the ***matrix D(4)*** . The ***shortest distances*** are saved in the ***matrix D(5).***



(a) Weighted graph

	v1	v2	v3	v4	v5
v1	0	1	8	9	5
v2	∞	0	7	9	19
v3	∞	8	0	6	16
v4	∞	10	2	0	10
v5	6	7	14	15	0

(b) Matrix $D^{(4)}$ stores shortest distances through v1,v2,v3,v4

	v1	v2	v3	v4	v5
v1	0	1	8	9	5
v2	25	0	7	9	19
v3	22	8	0	6	16
v4	16	10	2	0	10
v5	6	7	14	15	0

(c) $D^{(5)}$ tabulates shortest distances through v1,v2,v3,v4,v5

Shortest paths throuh: v1 v2 v3 v4 v5

Direct distance, v2->v1 = ∞
Distance, via v2->v5->v1 = 25

Direct distance, v3->v1 = ∞
Distance, via v3->v5->v1 = 22

Direct distance, v4->v1 = ∞
Distance, via v4->v5->v1 = 16

(d) Paths with shorter distances compared to direct distances

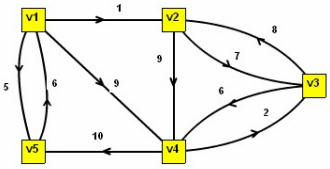
11

Floyd-Warshall Algorithm

Shortest Paths-I

- The ***shortest paths*** between the vertices ***v1,v2,v3*** and other vertices, ***via intermediate paths***, are shown in the figures. The paths are computed by using the ***order matrix P=[p_{ij}]*** .
- A ***zero*** entry in ***P*** indicates that there is ***direct path***. A ***non-zero*** value is the ***index*** of intermediate vertex. The indexes for intermediate vertices and the corresponding shortest distances, tabulated in matrix ***D***, are shaded red
- For example, an entry 5 in ***row 3, column 1***of *P* indicates that shortest path from ***v3*** to ***v1*** passes through vertex ***v5***.

(a) Weighted graph



(b) Shortest distances matrix D

	v1	v2	v3	v4	v5
v1	0	1	8	9	5
v2	25	0	7	9	19
v3	22	8	0	6	16
v4	16	10	2	0	10
v5	6	7	14	15	0

(c) Order P matrix

	v1	v2	v3	v4	v5
v1	0	0	2	0	0
v2	5	0	0	0	4
v3	5	0	0	0	4
v4	5	3	0	0	0
v5	0	1	2	1	0

5 indicates that shortest path between v3, v1 is via vertex v5

4 indicates that shortest path between v3, v5 is via vertex v4

(d) Shortest paths via intermediate vertices

v1->v3
Shortest disatance = 8,
Path: v1->v2 -> v3

v2->v1
Shortest disatance = 25,
Path: v2->v4 -> v5 -> v1

v2->v5
Shortest disatance = 19,
Path: v2->v4 -> v5

v3->v1
Shortest disatance = 22,
Path: v3->v4 -> v5 -> v1

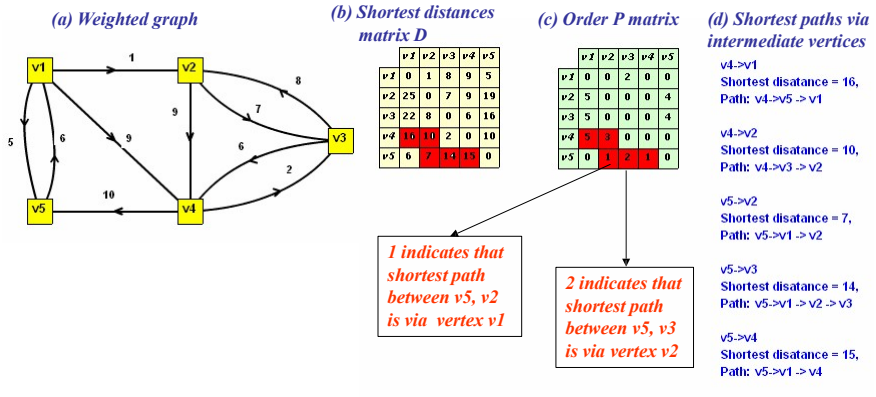
v3->v5
Shortest disatance = 16,
Path: v3->v4 -> v5

12

Floyd-Warshall Algorithm

Shortest Paths-2

- The ***shortest paths*** between the vertices ***v4,v5*** and other vertices , ***via intermediate vertices,*** are shown in the figures.



13

Floyd-Warshall Algorithm

Pseudo Code

As discussed, the results of intermediate computation of distances are stored into set of matrices. In actual implementation it is not necessary to maintain separate tables. Rather, the computed distances can be written back to the original table. In other words, the algorithm updates the entries after each iterative step. This procedure is implemented by the following pseudo code.

```
SHORTEST-DISTANCE(W, n) ►The weight matrix W of size n is the input
1  for i ←1 to n do
2     for j←1 to n do
3           D[i,j] ←W[i,j]                    ► Copy weight matrix to D
4  for k ←1 to n do                           ►Consider distances through vertices v1,v2,..vk
5     for i ←1 to n do
6        for j ←1 to n do
7           if D[i,k]+D[k,j] <D[i,j]          ► Select the shorter distance
8              then D[i,j] ←D[i,k]+D[k,j]
9  return D                                   ►Shortest distances are stored in table D
```

14

Floyd-Warshall Algorithm

Analysis

The code for the implementation of Floyd-Warshall algorithm consists of three nested loop, each executing n times Therefore, the ***time complexity***, $T(n)$, of the algorithm is

$$T(n)=\theta(n \times n \times n)=\theta(n^3)$$

- It follows that ***Floyd-Warshall*** algorithm is very efficient compared to brute force algorithm. The algorithm has ***space complexity*** of $\theta(n^2)$, because the computed table stores $n \times n$ distances.

15

Floyd-Warshall Algorithm

Listing Shortest Paths

- In order to store information about the shortest paths, an ***auxiliary table p_{ij}*** is used. The element p_{ij} holds the ***index of the intermediate vertex*** that was ***last examined*** while finding the shortest distance between the vertices v_i and v_j. If there was no intermediate vertex then p_{ij} is set zero.

- For example, if $p_{ij}=k$, then k is index of the vertex that was last examined before reaching vertex v_j. In other words, there was an intermediate path path v_k->v_j. Again if $p_{ik}=m$, then there was path from v_m to v_k, which is intermediate path between v_i and v_j .This path is

  $v_i..v$ ->v_k->v_j

- If there is no intermediate path between v_i and v_j then $p_{ij}=0$

- The table p_{ij} is created by modifying the code for the shortest path

16

Floyd-Warshall Algorithm

Pseudo Code

Algorithm with path matrix

```
SHORTEST-DISTANCE(W, n) ►The weight matrix  W of size n  is the  input

1   for i ←1 to n do
2     for j←1 to n do
3           D[i,j] ←W[i,j]                    ►  Copy weight matrix to D
4   for k ←1 to n do                          ►Consider distances through vertices v1,v2,..vk
5      for i ←1 to n do
6         for  j ←1 to n do
7           if D[i,k]+D[k,j] <D[i,j]          ►Select the shorter distance
8             then D[i,j] ←D[i,k]+D[k,j]
9                  P[i,j] ← k
10  return D,P                                ►Shortest distances are stored in table D
```

17